

ItemCase — Entwicklungsstand

Übergabe-Zusammenfassung für die Weiterentwicklung durch eine andere KI-Session

Erstellt: 17. September 2026 · Zeitraum: letzte ca. 2 Arbeitstage

Dieses Dokument fasst zusammen, was in den letzten Sitzungen an ItemCase (Electron-Desktop-App + Express/MySQL-Backend) umgesetzt wurde, welche Architekturentscheidungen getroffen wurden, wo die wichtigsten Dateien liegen, und was als Nächstes ansteht. Ziel: eine neue KI-Session soll ohne Rückfragen an die richtige Stelle im Code weiterarbeiten können.

1. Projektüberblick

ItemCase ist eine private Sammlungsverwaltungs-App (Electron/React Desktop-Client) für Karten, Spiele, Figuren, Comics, Münzen etc., die zu einer vollwertigen Social-/Community-Plattform mit eigenem Backend ausgebaut wurde.

- Client: Electron (Hauptprozess electron/main.js, Preload electron/preload.js, React-Renderer in src/, Vite-Build)
- Backend: eigenständiges Node/Express-Projekt in server/, MySQL (mysql2/promise), JWT-Auth, Socket.IO für Echtzeit
- DB-Host: mysql2e0e.netcup.net, Datenbank k194401_ItemCase, User k194401_Yoshi (Zugangsdaten in server/.env)
- Build: "npm run build" (Vite + electron-builder) erzeugt Windows-Installer in release/*.exe
- Standing Rule: vor Arbeitsbeginn immer "git pull", nach jedem Commit immer "git push" (ohne Rückfrage)

2. Was in den letzten 2 Tagen umgesetzt wurde

2.1 Backend-Fundament & Community (Phase 2)

- Eigener Server/MySQL aufgesetzt (server/), Schema in server/schema.sql, idempotente Migration via server/scripts/migrate.js
- Accounts, Login per Username ODER E-Mail, Freundschaftsanfragen per Username
- Private Sammlung vollständig in die Datenbank migriert (collection_items, collection_settings) — siehe Abschnitt 3

2.2 Phase 3: Gruppen, Chat, Benachrichtigungen, Forum

- Gruppen (collector_groups, group_members, group_bans) mit Rollen (owner/admin/moderator/member)
- Echtzeit-Chat über Socket.IO (server/src/realtime.js), Direkt- und Gruppenkonversationen + globaler Chat für alle Nutzer
- Notification Center (notifications-Tabelle + Live-Push per Socket.IO)
- Eigenständiges Forum (bewusst von Gruppen entkoppelt): 11 Kategorien, Likes/Kommentare, Trending, aktive Sammler, Wortfilter (blocked_words)
- Forum-UI komplett neu gestaltet (heller Feed + dunkle Sidebar nach Vorlage), doppelte Kategorie-Leiste oben entfernt (Kategorien nur noch in der rechten Sidebar)

2.3 Kritischer Bugfix: Private Sammlung pro Account

Bug: Gast-Login zeigte die private Sammlung des zuletzt eingeloggtten Accounts (eine einzige lokale collection.json-Datei unabhängig von der Identität). Behoben durch vollständige Migration der Sammlung in die Datenbank, pro Account (owner_id-Spalte), inkl. einmaliger automatischer Migration bereits vorhandener lokaler Daten (migrateLegacyLocalCollection() in electron/main.js) — kein Datenverlust bei echten Bestandsdaten.

2.4 UI-Überarbeitungen

- Sammlungsansicht (Hauptbereich) nach Mockup neu gestaltet, dabei theme-fähig gehalten (CSS-Variablen statt fixer Farben) und die linke Sidebar (Kategorien, Drag&Drop, Bilder) unverändert belassen
- Item-Karten überarbeitet: "î"-Menü für Bearbeiten/Löschen, farbcodierte Zustands-Badges

- Freundesliste und Chat-Fenster sind jetzt frei verschiebbare Panels (DraggablePanel.jsx, useDraggable-Hook, Position wird in localStorage gemerkt)
- Kategorie-Emojis werden nur noch in der Anzeige ausgeblendet (Daten behalten das Emoji), redundante Kategorie-Pillenreihe entfernt
- "Private Sammlung"-Badge neben dem Sammlungswert ergänzt

2.5 Phase 1 der "Differenzierungsfunktionen" (aus ItemCase_Gesamtkonzept_2026.pdf)

- Item-Status pro Sammlungsstück: Behalten / Doppelt / Tauschbar / Zu verkaufen / Suche besseres Exemplar (ownership_status-Spalte in collection_items), Badge auf der Karte, Filter in der Sammlungsansicht
- Wunschliste (wishlist_items-Tabelle, server/src/routes/wishlist.js, src/WishlistModal.jsx): privat by default, Priorität, gewünschter Zustand, Preisobergrenze, Notiz; automatische Benachrichtigung, wenn ein Freund ein gesuchtes Katalog-Item auf tauschbar/verkäuflich setzt (Server-seitiger Abgleich in collection.js beim Speichern eines Items)
- Foto-Upload: war bereits vorhanden (sharp-Pipeline entfernt EXIF/GPS automatisch) — keine Änderung nötig, bewusst KEINE KI-Texterkennung/Bildähnlichkeitssuche (das ist Phase 3 "Intelligenz" im Konzeptdokument, nicht umgesetzt)
- PDF-Sammlungsbericht-Export (data:exportPdf in electron/main.js, PDF via pdfkit direkt im Hauptprozess erzeugt, kein Server-Job-System): Umfang (ganze Sammlung/Kategorie), Bilder ein/aus, Preise ein/aus, Deckblatt + Zusammenfassung + Item-Liste

2.6 Intelligenter Excel/CSV-Import (eigenes Konzeptdokument, 3 Phasen)

Regelbasierte (keine KI) Spaltenerkennung für beliebige Excel/CSV-Dateien, auch ohne ItemCase-eigene Spaltennamen. Kern-Logik in src/smartImport.js, Wizard-UI in src/SmartImportModal.jsx, Datei-Lesen via neue IPC data:pickImportSpreadsheet in electron/main.js.

- Phase 1: Header-Erkennung, Spalten-Klassifikation (Name/Kategorie/Marke/Kennung/Zustand/Menge/Kaufpreis/Wert/Notiz) über Keyword- und Inhaltsmuster, eine Rückfrage bei mehrdeutigen Geldspalten, 4-Schritt-Wizard (Datei wählen !' Zuordnung bestätigen !' Vorschau !' Import), nichts wird ohne Bestätigung importiert
- Phase 2: gespeicherte Spalten-Zuordnungen (localStorage, per Spaltenstruktur-Signatur), Duplikaterkennung auch innerhalb derselben Datei, "Import rückgängig machen" direkt nach Abschluss
- Phase 3 (ohne KI): Summen-/Zwischenüberschriften-Zeilen werden erkannt und ausgeschlossen, automatischer Katalogabgleich über eindeutige Kennung (Auto-Verknüpfung) bzw. Name+Marke (nur als Vorschlag mit "Übernehmen"-Button)
- Explizit NICHT umgesetzt: optionale KI-Auswertung für schwer erkennbare Dateien — Nutzer hat sich bewusst dagegen entschieden (Kosten/Datenschutz bei Versand von Tabelleninhalten an einen externen Dienst)

2.7 Nebenbei behobener Bug

Notifications-Payload (JSON-Spalte in MySQL) wurde serverseitig immer als null zurückgegeben, weil mysql2 JSON-Spalten bereits automatisch parst und der Code zusätzlich JSON.parse() darauf angewendet hat (Absturz beim Parsen eines Objekts, stillschweigend abgefangen). Betraf ALLE Notification-Typen (auch Freundschaftsanfragen). Fix in server/src/routes/notifications.js: nur parsen, wenn der Wert tatsächlich ein String ist.

3. Wichtige Architektur-Entscheidungen & Muster

Bilder

Bilder werden bisher als base64-Data-URLs direkt in MySQL-MEDIUMTEXT-Spalten gespeichert (image_data), kein separater Dateispeicher. Das Konzeptdokument zu den Differenzierungsfunktionen empfiehlt für den Foto-Upload eigentlich externen WebP-Dateispeicher mit nur Pfaden in MySQL — diese Umstellung wurde NICHT vorgenommen (bewusste Abweichung, da der bestehende Foto-Upload mit EXIF-Stripping bereits funktional ausreichte). Sollte bei größerem Bild-Volumen überdacht werden.

Migrations-Regel (kritisch!)

In server/schema.sql NIEMALS "DROP TABLE" für Tabellen mit echten Daten verwenden — immer "CREATE TABLE IF NOT EXISTS" plus idempotente "ALTER TABLE ... MODIFY COLUMN" für Änderungen. Grund: ein früherer DROP+CREATE-Ansatz hat bei jedem Migrate-Lauf sämtliche Forum-Daten gelöscht. server/scripts/migrate.js toleriert ER_DUP_FIELDNAME/ER_DUP_KEYNAME als "bereits angewendet", nicht als Fehler.

Electron !" Server Anbindung

electron/main.js hat einen zentralen apiFetch()-Helper (Bearer-Token aus lokal gespeichertem JWT) und branched in fast jedem IPC-Handler auf loadAuthToken(): eingeloggt !' Server-API, sonst !' alte rein lokale Logik (Gast-Modus). remoteCollectionCall() ist ein Kurz-Wrapper für /collection-Endpunkte. Jeder neue Server-Endpunkt braucht i.d.R.: Route in server/src/routes/*.js, Mount in server/src/app.js, IPC-Handler in electron/main.js, Mirror in electron/preload.js (window.api.xxx), Aufruf aus src/*.jsx.

Realtime / Notifications

server/src/realtime.js verwaltet Socket.IO, joint jeden Client in einen Raum user:<id>, trackt Online-Status über ein onlineCounts-Map (nicht ein einfaches Set, wegen Multi-Device). server/src/utils/notify.js persistiert und pusht gleichzeitig eine Notification.

Browser-Preview-Workflow (für UI-Verifikation)

Für visuelle Tests neuer UI-Komponenten wird src/main.jsx temporär überschrieben (Mock von window.api, Rendern der einzelnen Komponente statt der ganzen App), per mcp Browser-Pane-Tools durchgeklickt, danach IMMER per "git checkout -- src/main.jsx" zurückgesetzt. Wichtig: Screenshots aus dem Browser-Pane sind für den Nutzer NICHT sichtbar — nur get_page_text/read_page verwenden, niemals einen Screenshot als "das ist jetzt die App" ausgeben (hat in der Vergangenheit zu Verwirrung geführt).

Rechtliche/Datenschutz-Vorgaben des Nutzers (unbedingt einhalten)

- KI-/Bildererkennung: nur eigene Modelle oder kommerziell lizenzierte APIs/Dienste, deren Bedingungen kommerzielle Nutzung erlauben. Hochgeladene Nutzerbilder dürfen nur mit klarer Upload-Lizenz verarbeitet werden.
- Wunschlisten, Tauschinteressen, Profile und Standortbezug sind personenbezogene Daten — nachvollziehbare Rechtsgrundlage, Datensparsamkeit, sichere Verarbeitung, Datenschutzerklärung nötig. Wunschliste ist deshalb standardmäßig privat implementiert.

4. Wichtigste Dateien (Kurzreferenz)

Server

server/schema.sql	– komplettes DB-Schema, idempotent
server/scripts/migrate.js	– wendet schema.sql an, toleriert "bereits angewendet"
server/src/app.js	– mountet alle /api/*-Routen
server/src/routes/collection.js	– private Sammlung (Items + Settings), Ownership-Status,
Wishlist-Match-Notify	
server/src/routes/wishlist.js	– Wunschliste CRUD
server/src/routes/forum.js	– Forum-Threads/Posts/Likes/Trending/Leaderboard
server/src/routes/friends.js	– Freundschaftsanfragen
server/src/routes/notifications.js	– Notification Center
server/src/realtime.js	– Socket.IO, Online-Presence
server/src/utils/notify.js	– Notification persistieren + live pushen

Electron

electron/main.js – zentraler IPC-Hub, apiFetch(), alle ipcMain.handle(...)
electron/preload.js – window.api.* Mirror aller IPC-Handler

Renderer (React)

src/App.jsx – Haupt-Komponente, sehr groß, State für fast alles
src/ItemForm.jsx – Item anlegen/bearbeiten, inkl. Ownership-Status
src/ItemCard.jsx – Karten-Darstellung inkl. Status-Badge
src/WishlistModal.jsx – Wunschliste-UI
src/SmartImportModal.jsx – 4-Schritt Import-Wizard
src/smartImport.js – reine Erkennungs-/Mapping-Logik (keine React-Abhängigkeit, gut
testbar)
src/ForumHubModal.jsx – Forum-UI (heller Feed + dunkle Sidebar)
src/DraggablePanel.jsx – useDraggable-Hook + Wrapper-Komponente
src/ChatWindow.jsx – einzelnes Chat-Fenster (jetzt frei verschiebbar)
src/i18n.jsx – DE/EN Übersetzungen, useI18n()/t(key, params)
src/styles.css – sehr groß, CSS-Variablen für Theming

5. Offene Punkte / Roadmap für die Weiterarbeit

5.1 Aus den Differenzierungsfunktionen (Phase 2 & 3 im Konzeptdokument)

- Duplikate/Tausch: echtes Matching zwischen Nutzern (TradeInterests-Tabelle), aktuell gibt es nur den Status und die Wunschlisten-Benachrichtigung, aber keine "Wer will das von mir?"-Übersicht
- Sammlungs-Timeline (automatisch generierte Zeitleiste der Sammlung) — komplett offen, nicht begonnen
- PDF-Bericht Ausbau: hübscheres Layout, mehr Optionen (aktuell nur Umfang/Bilder/Preise)
- Bild-Speicherung: Umstellung auf externen Dateispeicher (WebP + Pfad in MySQL) statt Base64-in-DB, falls DB-Größe/Performance zum Problem wird

5.2 Aus dem Excel-Import-Konzept

- Optionale KI-Auswertung — vom Nutzer bewusst abgelehnt (siehe Abschnitt 2.6), nicht ohne erneute Rückfrage umsetzen
- Mehrfach-Tabellenblatt-Import in einem Rutsch (aktuell: ein Blatt nach dem anderen)
- Persistente Import-Historie über Sitzungen hinweg (aktuell: Undo nur direkt nach einem Import in derselben Sitzung)

5.3 Aus dem großen Forum-Community-Konzept (nur teilweise umgesetzt)

- Gruppen-eigene Foren (als zusätzliches Feature zum bestehenden globalen Forum) — bewusst zurückgestellt, da der Nutzer Forum und Gruppen explizit entkoppelt haben wollte ("forum logik von gruppe trennen")
- Item-verlinkte Forum-Karten, Tags, "Hilfreiche Antwort"-Markierung, Moderationsrollen, Report-/Rate-Limit-System — aus einem sehr detaillierten Community-Spec, größtenteils als Roadmap markiert, nicht gebaut

5.4 Sonstiges

- server.pid wird manuell gepflegt (Windows Job-Control-PID-Mismatch bei nohup) — beim Server-Neustart immer mit "Get-NetTCPConnection -LocalPort 5100 -State Listen" die echte PID prüfen, bevor server.pid geschrieben wird
- Kein CI/Test-Automatisierung vorhanden — Verifikation bisher über manuelle smoke-test-*.js-Skripte in server/scripts/ gegen den lokal laufenden Server, danach SQL-Cleanup der Testdaten

6. Betrieb — schnelle Referenz

Server lokal starten

```
cd server && node server.js          # oder: node scripts/migrate.js zuerst, falls Schema geändert wurde
```

Renderer im Dev-Modus / Browser-Vorschau

```
npm run dev                          # startet Vite + Electron zusammen
```

Windows-Installer bauen

```
npm run build                        # vite build + electron-builder -> release/ItemCase Setup 1.0.0.exe
```

Git

```
git pull    (immer zuerst)          git push    (immer nach jedem Commit, ohne Rückfrage)
```